

Project Report: Cricbuzz LiveStats Analytics

Streamlit Live Demo: [Cricbuzz LiveStats Dashboard](#)

Domain: Sports Technology & Business Intelligence

Tech Stack: Python, RapidAPI (REST), MS SQL Server (SSMS), Pandas, Streamlit, Plotly

(a) Introduction

In the modern era of sports, data is as critical as athletic talent. The **Cricbuzz LiveStats Analytics** project is an end-to-end data pipeline and Business Intelligence (BI) dashboard designed to extract, transform, and visualize live cricket match data. By bridging real-time application programming interfaces (APIs) with an enterprise-grade relational database management system, this project converts deeply nested JSON payloads into actionable, executive-level insights. The resulting platform serves as a centralized hub for tracking match momentum, evaluating player performance through custom algorithms, and managing database records via a user-friendly web interface.

(b) Business Problem

Cricket generates an overwhelming volume of data per match, encompassing ball-by-ball events, player statistics, and team momentum shifts. However, raw data feeds are inherently difficult for decision-makers—such as team managers, broadcasters, and fantasy league operators—to consume directly.

The primary business challenges addressed by this project include:

1. **Data Silos and Formatting:** Real-time data exists in complex, nested JSON arrays that are not conducive to tabular querying or historical analysis.
2. **Lack of Instant KPI Calculation:** Stakeholders need immediate access to derived metrics (e.g., Strike Rates, Economy Rates, Form Categorization) rather than raw runs and wickets.
3. **Absence of Accessible Data Management:** Database administrators and analysts require a secure, code-free interface to execute CRUD (Create, Read, Update, Delete) operations to correct data anomalies instantly.
4. **Interactive Reporting:** Static reports are insufficient; users require dynamic filtering (e.g., by Country/Team) to isolate specific business questions and visualize trends seamlessly.

(c) Technical Executions

The project was executed through a robust ETL (Extract, Transform, Load) pipeline, culminating in an accurate dynamic frontend web application.

1. **Data Extraction (API Integration):** Utilized Python's requests library to interface with a Cricbuzz REST API hosted on RapidAPI. Securely fetched live match scorecards using structured GET requests and error-handling protocols for HTTP status codes.

2. **Data Transformation (Pandas):** Parsed the highly nested JSON response. Utilized `pd.json_normalize()` and the `.explode()` method to flatten player arrays, transforming hierarchical data into clean, normalized, two-dimensional DataFrames for both batting and bowling innings.
3. **Data Loading (SQL Server Backend):** Established an ODBC connection to a local MS SQL Server instance (PRACTICEDB) using SQLAlchemy and pyodbc. Loaded the cleaned DataFrames into dedicated relational tables (`match_batting_stats` and `match_bowling_stats`), effectively persisting the live data for historical querying.
4. **Analytical Processing (SQL Logic):** Developed comprehensive SQL queries ranging from beginner aggregations to advanced analytical functions. This included using INNER JOINS to identify all-rounders, Window Functions (like NTILE) for relative performance segmentation, and complex CASE statements to derive new categorical variables.
5. **Dashboard Development (Streamlit):** Built a multi-tab web application. Engineered a global sidebar filter that dynamically injects SQL WHERE clauses into the backend queries, allowing the entire dashboard to update based on the selected team. Integrated Plotly for interactive visuals and built a secure frontend form to execute SQL INSERT, UPDATE, and DELETE commands directly against the database.

(d) Interpretation of the Dashboard

The dashboard is designed to answer three questions at once: who batted best, who bowled most effectively, and how the match behaved overall. The charts suggest a match with one dominant batting performance, a mixed bowling effort, and enough wickets to keep the contest competitive.

- **Match Executive Summary:** The summary card gives the overall match picture: 544 total runs, 66 boundaries, 30 wickets, and an average strike rate of 115.74. That combination suggests a reasonably high-scoring match with active batting and a fair amount of wicket-taking. The raw data tables below the KPIs confirm that the dashboard is pulling directly from the underlying match dataset rather than showing only polished visuals.
- **Batting Analytics:** This screen highlights runs vs. strike rate and boundary distribution. One player has a very strong combination of high runs and high strike rate around the 150+ range, which suggests an aggressive, match-defining innings. The boundary chart shows that some players contributed mostly through fours, while sixes are fewer and more unevenly distributed. The reason it matters is that it distinguishes between a batter who accumulated runs steadily and one who rapidly accelerated the innings.
- **SQL Analytics (Top Run Scorers):** This view summarizes the highest batting contributions derived from direct SQL queries. One batter stands far above the rest with 84 runs, while the next group is much lower at around 27, 26, and 22 runs. This large gap between the top bar and the rest indicates the innings was heavily driven by one major score rather than evenly spread batting contributions across the roster.
- **Bowling Analytics:** This view compares bowling economy and wicket impact. The most economical bowlers are hovering around 4.0 to 6.2 economy, while one bowler is

visibly expensive at about 11.3. The scatter plot adds crucial context: bowlers with lower runs conceded and some wickets are the most effective, whereas bowlers with high runs conceded but few wickets are less efficient. The panel is not just showing wickets; it is showing the vital trade-off between control and breakthroughs.

- **Database Administrator / Data Management View:** This screen is built for editing player match records in real time. With the selected operation set to "Update," the form allows an admin to change a batter's runs, balls faced, fours, and sixes. This acts as a true admin panel by explicitly stating that changes will execute live SQL queries and refresh the dashboard immediately. Furthermore, the dropdown on the left ensures the data can be accurately filtered by team before making administrative edits.

(e) Managerial Implications

The deployment of this dashboard offers significant strategic advantages for various stakeholders in the sports business ecosystem:

1. **Strategic Team Management:** Coaches and selection committees can utilize objective performance visuals to make data-backed decisions regarding squad rotation, dropping underperforming players, or promoting high-impact batsmen who show strong strike-rate combinations.
2. **Media and Broadcasting Enhancements:** Commentary teams require instant, digestible insights. The dashboard's ability to filter dynamically by team and instantly generate visuals (like the Boundary Distribution) provides broadcasters with immediate talking points and on-screen graphic data.
3. **Fantasy Sports and Betting Analytics:** In highly monetized sectors like fantasy cricket, users rely on identifying undervalued assets. The visualization of bowling economy versus wickets taken helps users spot players contributing efficiently, maximizing point generation.
4. **Data Governance & Operational Efficiency:** The inclusion of the CRUD Data Management tab empowers non-technical staff (like data entry operators or match officials) to maintain database integrity. If the API misses a boundary or mis-attributes a stat, authorized personnel can correct the record directly through the UI, ensuring the downstream analytics remain 100% accurate without requiring a backend engineer.

Link-

https://rapidapi.com/cricketapilive/api/cricbuzz-cricket/playground/apiendpoint_38fe7ec9-e3ba-4d9a-b938-c6c5836c36ad